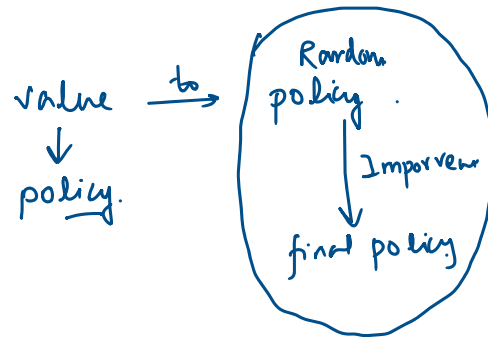
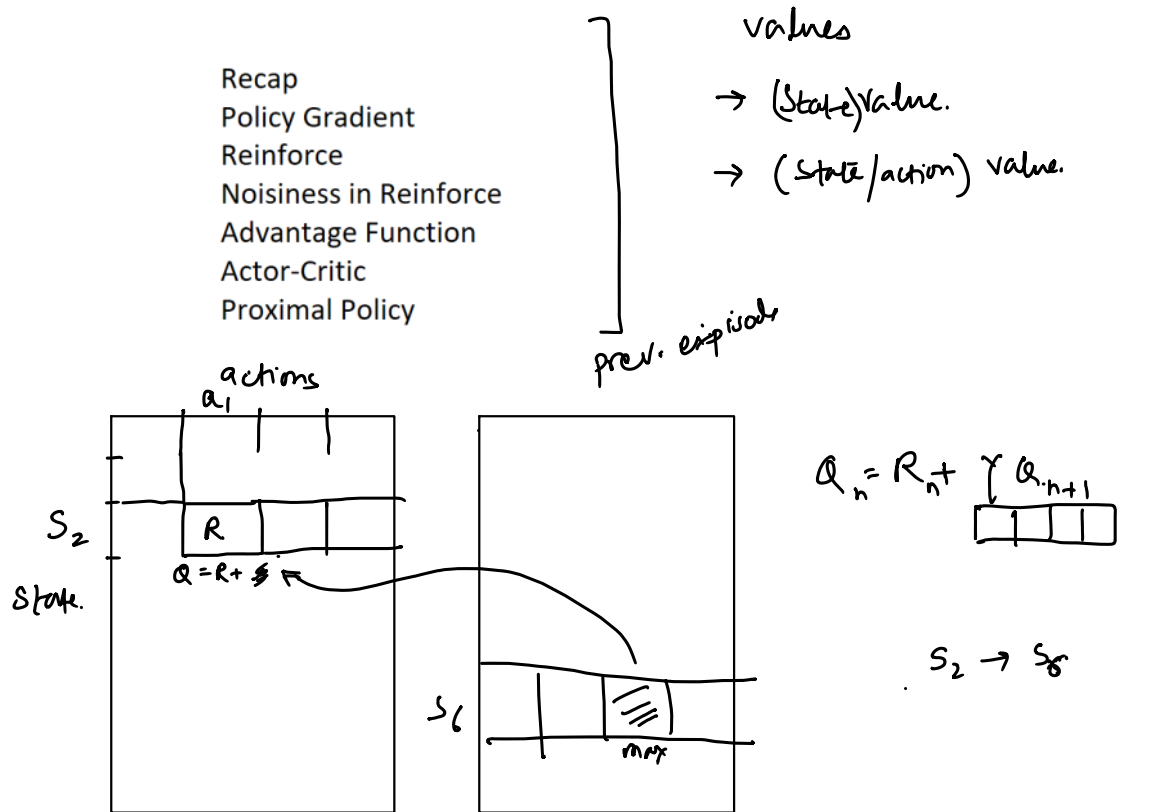
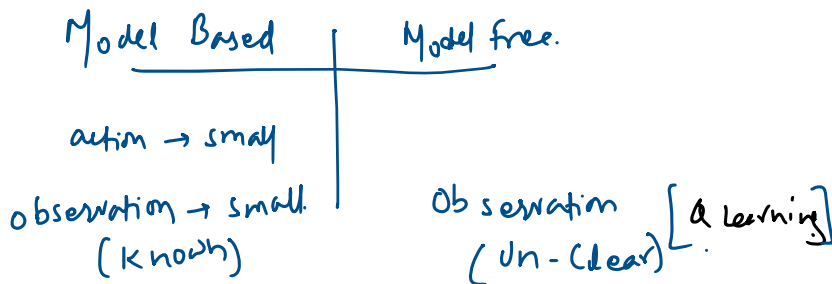


# Policy Gradient

12 September 2026 23:44



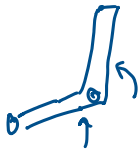
large action space  
or  
continuous action space.



action space is small

↓ action (small)  
observation (bigger / continuous)  
(DQN)

Action Space is Bigger or Continuous  
(Policy Gradient)



Action Space is Bigger or Continuous

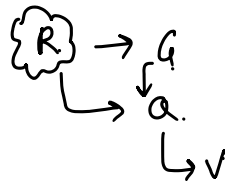
policy (policy gradient)

↳ REINFORCE ✓

↳ Advantage function

↳ Actor-critic [checks-balances]  
policy value

↳ proximal policy optimization (step based checking) (2nd layer)



|   |   |   |    |
|---|---|---|----|
| 1 | 1 | 1 |    |
|   | 2 |   | 2  |
|   | 2 |   |    |
|   |   |   | 10 |

$G_5$  = total Reward  
starting from 5 to last spot

$$G_5 = R_6 + \gamma R_7 + \gamma^2 R_8 + \dots + R_L$$

$$= R_6 + \gamma [R_7 + \gamma R_8 + \gamma^2 R_9 + \dots + R_L]$$

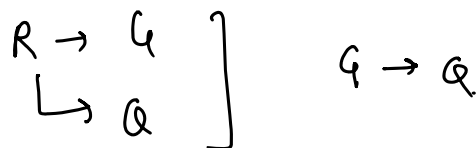
$$G_5 = R_6 + \gamma G_6$$

$$\downarrow$$

$$Q_5 = R_6 + \gamma Q_6$$

$$Q_k = R + \gamma Q_{k+1}$$

$G$  is just the sample of  $Q$



$Q_{10}$



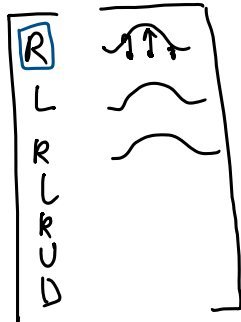
$$V_{max} = \max(Q_{10})$$

$$\neq \max(Q_{10})$$

$$V_{avg} = \text{avg}(Q_{10}) \rightarrow \pi \times Q_{a_1} + \pi Q_{a_2} + \pi Q_{a_3} + \pi Q_{a_4}$$

policy  $\rightarrow ?$

Set of actions



$\uparrow$   
 $\leftarrow S_{10} \rightarrow$

$a_L = 0.3 \checkmark$   
 $a_R = 0.4 \checkmark$   
 $a_U = 0.1 \checkmark$   
 $a_D = 0.0 \checkmark$

$a_L \rightarrow 1/3$   
 $a_R \rightarrow 1/3$   
 $a_U \rightarrow 1/3$

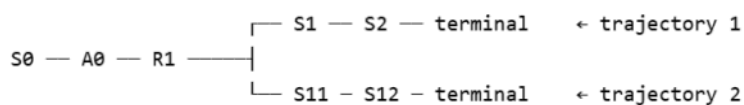
$\rightarrow$  policy  $\bar{\pi}$

$\bar{\pi}(a|s) \rightarrow$  probabilistic

$$V_{avg} = \bar{\pi}(a_1|s_{10})Q + \bar{\pi}(a_2|s_{10})Q + \bar{\pi}(a_3|s_{10})Q + \bar{\pi}(a_n|s_{10})Q$$

$$V_{avg} = \sum \bar{\pi} \times Q$$

Trajectory



$$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 \quad (S_0, A_0, G_0)$$

$$G_1 = R_2 + \gamma R_3 \quad (S_1, A_1, G_1)$$

$$G_2 = R_3 \quad (S_2, A_2, G_2)$$

$$\theta \leftarrow \theta + \alpha \boxed{G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)} \leftarrow \text{policy Gradient}$$

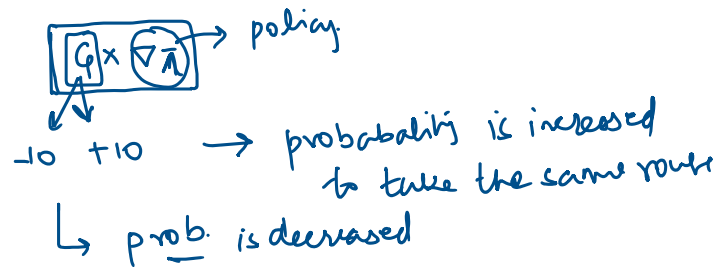
$$G_t \nabla \log \pi = 4$$

$$G_t \nabla \log \pi = 7 \quad E[\cdot] \approx \frac{4 + 7 + 1}{3} = 4$$

$$G_t \nabla \log \pi = 1$$

$$\frac{1}{N} \left[ G_0^{(1)} \nabla \log \pi(A_0^{(1)} | S_0^{(1)}) + G_1^{(1)} \nabla \log \pi(A_1^{(1)} | S_1^{(1)}) + \dots \right]$$

policy Gradient



increment in prob of different policy

policy is going to improved by  $H/N$ .

change is significant in policy

(Noisiness)  
High variance  $\rightarrow$  problematic at Inference.

$$\boxed{G} \times \nabla \text{pol.}$$

$$\downarrow$$

$$\boxed{V_{avg}} \times \nabla \text{pol.}$$

$$G \approx Q$$

$$\begin{bmatrix} q_1 & \boxed{q_2} & q_3 & q_4 \end{bmatrix} = \text{avg } V$$

$\pi_1 \quad \pi_2 \quad \pi_3 \quad \pi_4$

$$V_{avg} = \pi_1 q_1 + \pi_2 q_2 + \pi_3 q_3 + \pi_4 q_4$$

$$V_{\text{avg}} = \pi_1 q_1 + \pi_2 q_2 + \pi_3 q_3 + \pi_4 q_4$$

$$(Q - V) \times \nabla$$

$$\text{Adv} = \underbrace{Q - V}_d = \begin{matrix} +ve \\ -ve \end{matrix}$$

$$\text{Adv. Function} = Q_{\text{Spec}} - V_{\text{Avg}}$$

$$(\text{Reforce}) \text{ Adv. fm} = Q - V$$

policy Gradient ( $\nabla \text{pol.}$ )

$$p = p + \nabla p$$

$$\theta = \theta + \boxed{\nabla p} \rightarrow \boxed{G \times \nabla p}$$

$G \times \nabla \text{pol} \rightarrow \text{Noisy Improvement}$

↓

$$V \times \nabla \text{pol}$$

↓

$$A \times \nabla \text{pol} = (Q - V) \nabla \text{pol.}$$

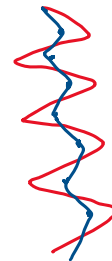
REINFORCE + Adv. function  $\rightarrow$  REINFORCE with Advantage.

Policy

Actor Network  
↓  
 $\pi$

Value

Critic Network  
↓  
(Vs)

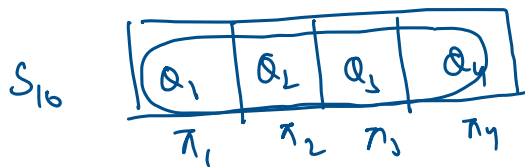


$$\boxed{V_{\text{avg}}} = \pi_1 \times Q_1 + \pi_2 Q_2 + \pi_3 Q_3$$

→ train  $\boxed{V_{\text{avg}}}$

ob  $\rightarrow$  class [  $\text{func 1} \rightarrow \pi \rightarrow \text{parameter + lr}$   
 $\text{func 2} \rightarrow v \rightarrow \text{parameters + lr}$  ]

$m_{\text{pol}} \rightarrow \text{ob.1}()$   
 $m_v \rightarrow \text{ob.2}()$



$$V_{\max} = \max(S_0)$$

$$\frac{Q_1 + Q_2 + Q_3 + Q_4}{4} \leftarrow$$

$$V_{\text{avg}} = Q_1 \pi_1 + Q_2 \pi_2 + Q_3 \pi_3 + Q_4 \pi_4$$

$$A_{\text{avg}} = \frac{1}{4} Q_1 + \frac{1}{4} Q_2 + \frac{1}{4} Q_3 + \frac{1}{4} Q_4$$

$$= 0.5 Q_1 + 0.5 Q_2 + 0.5 Q_3 + 0.5 Q_4$$

$$\sqrt{A_{\text{avg}}} = 0.1 Q + 0.5 Q_1 + 0.9 Q_3 + 0.2 Q_4$$

# Policy gradient update

```
loss = []
for log_prob, Gt in zip(log_probs, returns):
    loss.append(-log_prob * Gt)
loss = torch.stack(loss).sum()
```

loss.backward()  $\rightarrow \nabla \text{loss}$

$\sum G \times \log(\text{policy}) \rightarrow \text{Optimization}$

$$\text{Loss} = - \frac{\text{Optimization}}{\log} = - \sum G \times \log(\text{policy})$$

$$G_{L-1} = R_{L-1} + R_L$$

$$G_3 = R_3 + \dots + R_L$$

$$G_2 = R_2 + \dots + R_L$$

$$G_1 = R_1 + \dots + R_L$$

$$G_{L-2}$$

$$G_{L-3}$$

$$\begin{aligned} & \boxed{G - V} \rightarrow \text{More controlled} \\ & = A \begin{cases} \nearrow +ve \\ \searrow -ve \end{cases} \end{aligned}$$

$$\rightarrow V(S) = 10$$

$$\rightarrow Q(S, A) = 14$$

$$A(S, A) = 14 - 10 = +4$$

Action was better than expected

↓

Advantage = +4

↓

Increase probability of this action

REINFORCE with Advantage.

$$\begin{aligned} & G \times \text{pol} \\ & \downarrow \\ & \textcircled{A} \times \text{policy} \rightarrow A = (G - V) \end{aligned}$$

```
# Compute Advantages -> advantage function A(s, a) = R - V(s)
def compute_advantages(rewards, values, gamma=0.99):
    discounted_rewards = []
    R = 0
    for r in reversed(rewards):
        R = r + gamma * R
        discounted_rewards.insert(0, R)

    discounted_rewards = torch.tensor(discounted_rewards, dtype=torch.float32)
    advantages = discounted_rewards - torch.tensor(values, dtype=torch.float32, requires_grad=True)
    return advantages, discounted_rewards
```

$$G - V$$

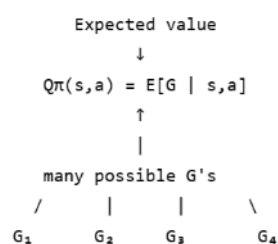
Actor:  $\pi_{\theta}(a|s)$

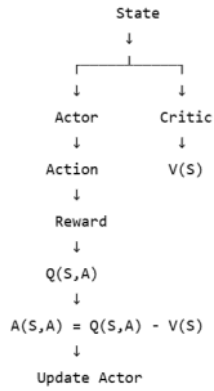
Critic:  $V_{\phi}(s)$

Typical Actor-Critic:

$S \rightarrow \text{Actor} \rightarrow \pi(a|s)$

$S \rightarrow \text{Critic} \rightarrow V(s)$





# Policy Network

```

class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_dim, 32)
        self.fc2 = nn.Linear(32, action_dim)

```

```

def forward(self, state):
    x = torch.relu(self.fc1(state))
    return torch.softmax(self.fc2(x), dim=-1)

```

$\leftarrow \text{prob} = \pi(a|s)$

# Value Network (Baseline)

```

class ValueNetwork(nn.Module):
    def __init__(self, state_dim):
        super(ValueNetwork, self).__init__()
        self.fc1 = nn.Linear(state_dim, 32)
        self.fc2 = nn.Linear(32, 1)

```

```

def forward(self, state):
    x = torch.relu(self.fc1(state))
    return self.fc2(x) # Outputs V(s)

```

$\leftarrow \text{Average (Regression)} = V_{avg}$

```

done = False
while not done:
    state_tensor = torch.tensor(state, dtype=torch.float32)
    value = value_net(state_tensor)
    # Compute V(s)
    action_probs = policy(state_tensor) # Get pi(a|s)
    dist = Categorical(action_probs)
    action = dist.sample() → 1
    log_prob = dist.log_prob(action)
    log_probs.append(log_prob)
    values.append(value)

```

next\_step → state, reward, done, \_ = env.step(action.item())  
rewards.append(reward)

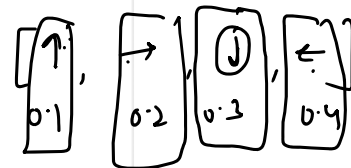
# Compute Advantage Function A(s, a)

```

advantages, discounted_rewards = compute_advantages(rewards, values, gamma=0.99)

```

env.step(action)



$e_1 \rightarrow 0.4$   
 $e_2 \rightarrow 0.4$   
 $e_3 \rightarrow 0.3$   
 $e_4 \rightarrow 0.1$   
 $e_5 \rightarrow 0.2$

REINFORCE  
↓ (Actor)

```

policy_loss = -torch.stack(log_probs)
policy_loss = policy_loss.mean()

```

```

# Update Policy Network
policy_optimizer.zero_grad()
policy_loss.backward()
policy_optimizer.step()

```

(Critic)

```

value_loss = (torch.stack(values).squeeze() - discounted_rewards) ** 2
value_loss = value_loss.mean()

```

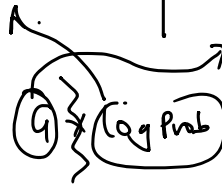
```

# Update Value Network
value_optimizer.zero_grad()
value_loss.backward()
value_optimizer.step()

```

```
policy_loss.backward()
policy_optimizer.step()
```

$\pi$



```
value_loss.backward()
value_optimizer.step()
```

$\nabla_{\log}$

$$A = G - V$$

LM  $\rightarrow$  [Action Space]

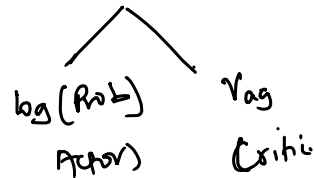
$\pi(a|s) \rightarrow \text{sample}()$

```
Episode 0, Reward: 62.0, Value Loss: 848.0458
Episode 100, Reward: 236.0, Value Loss: 1415.0114
Episode 200, Reward: 500.0, Value Loss: 794.8258
Episode 300, Reward: 441.0, Value Loss: 303.3233
Episode 400, Reward: 500.0, Value Loss: 476.7798
Episode 500, Reward: 108.0, Value Loss: 270.5582
Episode 600, Reward: 500.0, Value Loss: 483.9936
Episode 700, Reward: 500.0, Value Loss: 584.4250
Episode 800, Reward: 500.0, Value Loss: 561.9401
Episode 900, Reward: 500.0, Value Loss: 423.2528
```

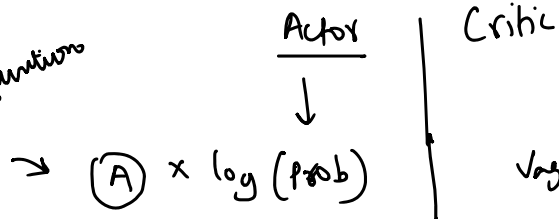
REINFORCE

$G \times \log(\text{Prob})$

with Advantage



Advantage function



```
# Actor Loss (Policy Gradient)
actor_loss = -torch.sum(torch.stack(log_probs) * advantage.detach())

# Critic Loss (Mean Squared Error)
critic_loss = torch.nn.functional.mse_loss(values, returns)

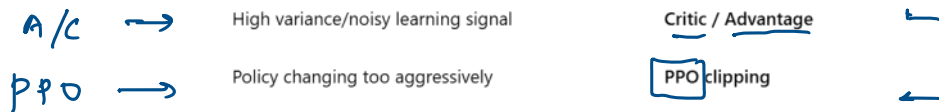
# Optimize Actor and Critic together to retain the graph
actor_optimizer.zero_grad()
critic_optimizer.zero_grad()

# Combine losses to compute gradients only once
total_loss = actor_loss + critic_loss
total_loss.backward() # Backpropagate through the combined loss

actor_optimizer.step()
critic_optimizer.step()
```

$$a = a + \nabla$$

$$C = C + \nabla$$



$$r_t = \frac{\pi_{\text{new}}(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

$$\text{ratio} = \frac{\text{new. policy}}{\text{old. policy}}$$

$$r_t \rightarrow \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$$

$$\epsilon = 0.2$$

$$\min(\text{Ratio}, \text{Clamp}[\text{Ratio}, \text{min}, \text{max}])$$

G x policy  
 [A] x policy  
 [R x A] x policy.

```

action_probs, value = policy(state)

action = torch.multinomial(action_probs, 1).item() # Sample action
reward = np.random.rand() # Fake reward
next_state = torch.rand(state_dim) # Fake next state

# Compute Advantage
_, next_value = policy(next_state)
advantage = reward + gamma * next_value - value

# Policy ratio
new_probs, _ = policy(state)
r_t = new_probs[action] / action_probs[action]

# PPO Clipped Loss
loss = -torch.min(r_t * advantage, torch.clamp(r_t, 1 - epsilon, 1 + epsilon) * advantage).mean()
  
```

$$[R, \text{Clamp}(R, \text{min}, \text{max})]$$

$$[R] \times A$$

$$\text{torch.clamp}([-10, 5, 9, 20], \text{min} = -7, \text{max} = +10)$$

↓

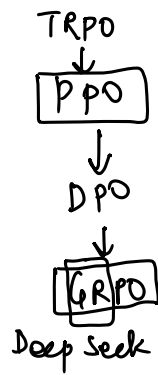
$$-7, 5, 9, 10$$

Application

- Intelligent Drone
- Drones
- LLMs

- Critical Application  
Medical Applications.

- ① Frozen-lake (
  - ② Cart-pole (Action Space is discrete) ←
  - ⋮
  - ③
    - Ping/Pong /
    - Ant
    - Humanoid.
    - .
- } Action Space (large/continuous)



$$P_1 / P_2 / P_3 / P_4$$

Avg

Policy Grad.



REINFORCE

$G \times \text{policy}$

Advantage Function  $[G - V_{\text{avg}}] \times \text{policy}$

REINFORCE  
with  
Adv.

Actor/Critic  
[policy] [value]

PPO

$$\text{Ratio} = \frac{\text{New Pol}}{\text{Old Pol}}$$

Ratio need damped